

India / UK / USA — Frontend & Backend Architecture

Full-stack breakdown: file structure and functional structure, per jurisdiction, plus the shared foundation every jurisdiction sits on.

Part 1 — Shared Backend Foundation

Before the per-country detail: one data model, one resolution function, and one context object are shared by all six supported jurisdictions (IN/UK/US/AU/CA/DE). Understanding these once explains most of what looks "magic" in any single country's file.

1.1 — The canonical / org-scoped data model

TABLE	PURPOSE
JurisdictionPack	Versioned identity for one Tax or Policy pack — <code>pack_id</code> , <code>jurisdiction_country</code> / <code>jurisdiction_state</code> , <code>status</code> (Draft→Active→Retired), <code>pack_type</code> ("tax" "policy").
ContributionRate	One statutory rate row (PF/ESI/PT/SS/NI/...). <code>organization_id IS NULL</code> = canonical (Super-Admin-owned, linked to a JurisdictionPack); <code>organization_id</code> set = that org's own synced copy — what the engine actually reads at calculation time.
TaxSlab	Same canonical/org split, for bracket-shaped data (income tax bands, PT_FLAT brackets, UK NI_BAND rows, India SURCHARGE tiers).

Two hardcoded fallback layers exist below this DB model: (1) `_CONTRIBUTION_RATES_BY_COUNTRY` / `_TAX_SLABS_BY_COUNTRY` seed dicts in `service.py` — used to seed an ORG's own rows the first time its payroll runs with none cached; (2) per-parameter Python constants in each `engine/countries/*.py` file, passed as the `default` to `resolve_jurisdiction_parameter()` . Both are now browsable read-only via Super Admin Compliance's "Engine Fallback Defaults" page.

1.2 — `resolve_jurisdiction_parameter()` — the one central resolver

`backend/app/modules/payroll/engine/countries/shared.py` — every country calculator calls this instead of reading a constant directly:

```
resolve_jurisdiction_parameter(rate_map, key, default, side=None, country=None)
1. Look up `key` in rate_map (built from ContributionRate rows the caller already resolved)
2. If a configured row exists for `side` (employee/employer) or as a flat amount -> use it
3. Otherwise -> use `default` (the engine-only hardcoded constant), logging a warning
   so a compliance gap is visible in logs, never silently invisible
```

1.3 — `PayrollContext` — the shared input every calculator receives

`backend/app/modules/payroll/engine/base.py` — one dataclass, built once per employee per payroll run, passed to whichever country's `calculate(ctx)` function applies. Country-specific fields are opt-in (None/empty = "not set," never inferred): `tax_code` / `ni_category` (UK), `tax_regime` (India), `study_loan_plan` / `study_loan_balance` (UK+AU shared mechanism), `church_tax_liable` (Germany), `employer_tax_profiles` / `reciprocity_suppresses_work_state` / `locality_rate` (US-only).

1.4 — Shared `service.py` + API layer

LAYER	ROLE
<code>backend/app/modules/payroll/service.py</code>	<code>list_canonical_contribution_rates</code> / <code>upsert_canonical_contribution_rate</code> / <code>upsert_canonical_tax_slab</code> (Super Admin write path) · <code>get_contribution_rates</code> / <code>get_tax_slabs</code> (org read path, seeds on first empty read) · <code>sync_org_rates_from_canonical</code> / <code>assign_pack_to_organizations</code> (the "push canonical → org" action) · <code>_resolve_effective_rate_inputs</code> (the live canonical-read gate for opted-in orgs)
<code>backend/app/modules/super_admin/router.py</code>	Every <code>/api/super-admin/compliance/*</code> endpoint — thin, delegates straight into <code>payroll_service</code> / <code>sa_service</code> functions. Identical shape for all 6 countries; nothing country-specific here.
<code>backend/app/modules/payroll/engine/tax_resolver.py</code>	<code>resolve_tax_configuration</code> / <code>get_state_scoped_config</code> — resolves which JurisdictionPack applies for a (country, state, regime, date), winner-take-all at country level with an additive state-layer fallback (so a PT-only state pack doesn't wipe the country's income-tax brackets).

Part 2 — India (IN)

2.1 — Frontend file structure

FILE	ROLE
<code>pages/JurisdictionCompliance/INCompliancePage.jsx</code>	Everything India-specific in one file: <code>ParametersTab</code> (87A/cess/surcharge/retirement), <code>PTSlabsTab</code> / <code>PTSlabFormModal</code> (state PT brackets), <code>TDSBracketsTab</code> / <code>TDSBracketFormModal</code> (country-level income-tax brackets, regime-aware), <code>IndiaComplianceConfig</code>
<code>components/jurisdiction/india/*</code> (5 files)	<code>inComponentConfig.js</code> (classification map + <code>sanitizeNumeric()</code>), <code>INComponentPickerModal.jsx</code> , <code>INComponentFormModal.jsx</code> , <code>INComponentCard.jsx</code> , <code>INTaxComponentsTab.jsx</code> — the business-language "Contribution Components" picker (PF/ESI/PT/TDS)

2.2 — Backend file structure

FILE	ROLE
<code>engine/countries/india.py</code>	The entire India calculation — <code>calculate(ctx)</code> , <code>_calculate_annual_tax_in</code> , <code>_apply_section_87a_rebate</code> , <code>_apply_surcharge</code> , <code>_apply_cess</code> , <code>_resolve_state_pt_bracket</code>
<code>service.py</code> — India-specific slices	<code>_CONTRIBUTION_RATES_BY_COUNTRY["IN"]</code> (<code>pf/esi/pt/tds/esi_wage_ceiling/standard_deduction/rebate_87a_limit/rebate_87a_max</code>), <code>_TAX_SLABS_BY_COUNTRY["IN"]</code> (7 MARGINAL_RATE brackets)

2.3 — Functional structure: what `calculate(ctx)` actually does

```
1. PF = Basic x employee/employer rate_pct (rate_map["pf"], no wage ceiling – known gap)
2. ESI = Gross x employee/employer rate_pct, only if Gross <= esi_wage_ceiling (default ₹21,000)
3. PT = _resolve_state_pt_bracket(state_slabs) if a real state PT_FLAT bracket exists (e.g. Telangana)
      else flat rate_map["pt"] (country-level fallback)
4. TDS = _calculate_annual_tax_in(annual_gross, slabs, rate_map, tax_regime):
      a. standard_deduction subtracted (New: ₹75,000 default / Old: ₹50,000 default – genuinely
         different code-level defaults, not a bug)
      b. progressive MARGINAL_RATE brackets summed (shared._calculate_annual_tax)
      c. Section 87A rebate applied, with marginal relief capping the cliff-edge (_capped_marginal_amount)
      d. Surcharge applied above configured SURCHARGE tiers, own marginal relief
      e. 4% Health & Education Cess on (tax + surcharge)
      tds = (annual_tax + surcharge + cess) / 12 – the FULL combined monthly liability
```

Regime selection (New/Old) is per-employee (`ctx.tax_regime`) — the SAME pack can hold both regimes' brackets/parameters side by side, each row tagged `tax_regime="New" | "Old"`, resolved per employee at calculation time. This is exactly what the Tax Parameters tab's New/Old toggle and the TDS Brackets tab's regime toggle are each independently managing.

Part 3 — United Kingdom (UK)

3.1 — Frontend file structure

FILE	ROLE
<code>pages/JurisdictionCompliance/UKCompliancePage.jsx</code>	Everything UK-specific: <code>StatutoryThresholdsTab</code> , <code>NICategoriesTab</code> , <code>WorkplacePensionTab</code> (+2 sub-views), <code>StudentLoansTab</code> , <code>PAYESlabsTab</code> , <code>ukComplianceConfig</code>
<code>components/jurisdiction/ComplianceConfigModal.jsx</code>	One shared, type-aware Add/Edit modal (5 configTypes) — UK-only in practice despite living in the shared folder

3.2 — Backend file structure

FILE	ROLE
<code>engine/countries/uk.py</code>	<code>calculate(ctx)</code> , <code>interpret_tax_code</code> (HMRC tax-code parsing), <code>_calculate_annual_tax_uk</code> , <code>_resolve_ni_bands</code> / <code>_calculate_ni_from_bands</code> , <code>_resolve_pensionable_pay</code>
<code>service.py</code> — UK-specific slices	<code>_CONTRIBUTION_RATES_BY_COUNTRY["UK"]</code> (national-insurance/employer-pension/personal_allowance/pa_taper_threshold/ni_primary_thresh/ni_upper_threshold/ni_secondary_thresh/ni_upper_rate), <code>_TAX_SLABS_BY_COUNTRY["UK"]</code> (4 brackets), <code>resolve_uk_configuration</code> (picks sub-jurisdiction by tax-code prefix, never worksite)

3.3 — Functional structure: what `calculate(ctx)` actually does

- National Insurance:
 - If `NI_BAND` TaxSlab rows exist for `ctx.ni_category` -> banded calc (`_calculate_ni_from_bands`)
 - Else -> flat Primary/Upper rate split at `ni_primary_thresh/ni_upper_threshold` (employee), Secondary threshold (employer) – genuinely different mechanisms, same fallback shape as India's `PT_FLAT`-vs-flat-rate pattern
- Workplace Pension: employer % always active; employee % stays 0 unless explicitly configured (never silently starts deducting from an existing org's next payslip)
- PAYE:
 - `interpret_tax_code(ctx.tax_code)` parses standard codes (1257L), K-codes (negative allowance, untaxed benefit), 0T, NT, and flat-rate families (BR/D0/D1, Scottish SD0-3, Welsh CD-codes)
 - Personal Allowance tapers above £100,000 (halved reduction)
 - Sub-jurisdiction (Scotland/Wales/NI) brackets used INSTEAD of national ones when configured – resolved upstream by `service.py`, never compared by worksite inside `uk.py` itself
- Student/Postgraduate Loan: deducted whenever `ctx.study_loan_balance > 0` and a plan is set – 5 plans (Plan 1/2/4/5/Postgrad), each its own threshold+rate

Part 4 — United States (USA)

4.1 — Frontend file structure

The most layered of the three — a page-level orchestrator plus two separate component folders.

FILE	ROLE
<code>pages/JurisdictionCompliance/USACompliancePage.jsx</code>	Top-level section switcher (Overview/Federal/State-District/Local/SUI/Reciprocity/Source Evidence/Organizations/Versions/Audit) + <code>?section=</code> URL scheme
<code>.../components/usa/USTaxPackWorkspace.jsx</code> + 5 more files	USA's own promoted Organizations/Versions/Audit views, dashboard, and state hooks
<code>config/jurisdictions/usaComplianceConfig.jsx</code>	The extraTabs/hiddenTabs object passed into JurisdictionLayout — USA's equivalent of India's inline config, pulled into its own file
<code>components/jurisdiction/usa/*</code> (9 files)	<code>usaComponentConfig.js</code> , the Tax Components picker/form/card/tab (the pattern India's own picker was built to mirror), Income Tax Brackets tab + filter bar, <code>USANewPackModal.jsx</code>
<code>components/jurisdiction/{Sui,Reciprocity,Locality,SourceEvidence}...Panel.jsx</code>	4 page-level panels — USA-only in practice, live in the shared folder

4.2 — Backend file structure

USA is also the deepest jurisdiction on the backend — 5 dedicated tables beyond the shared JurisdictionPack/ContributionRate/TaxSlab model, all purely additive (nullable FKs into existing tables, every non-US row unaffected):

TABLE / FILE	ROLE
<code>engine/countries/us.py</code>	<code>calculate(ctx)</code> , <code>_calculate_annual_tax_us</code>
<code>SourceArtifact</code>	One row per official publication a statutory value came from (agency, title, checksum, reviewer approval) — the audit-grade evidence trail
<code>LocalityDataset</code> + <code>LocalityRate</code>	Versioned, staged/approved-as-a-unit import of county/municipal/school-district tax codes — thousands of rows per state, not hand-typed one at a time
<code>EmployerTaxProfile</code>	Tenant-specific, agency-assigned SUI rate — deliberately NOT a ContributionRate (an org's own policy choice) since a SUI rate is a statutory fact assigned by a government agency, never inferred from prior deductions
<code>ReciprocityRule</code>	Cross-state reciprocity agreements (resident vs. work state) — referenced by <code>_resolve_us_reciprocity</code>
<code>TaxabilityRule</code> + <code>EmployeePreTaxDeductionElection</code>	Per-earning-type × per-tax-component taxability (e.g. a 401(k) deferral exempt from federal income tax but not Social Security) — platform-wide table, seeded <code>is_taxable=True</code> everywhere so no country's math changes until a real differentiated row is added

4.3 — Functional structure: what `calculate(ctx)` actually does

```
1. Social Security = min(annual_gross, ss_wage_base) x employee/employer rate (both configurable)
2. Medicare       = annual_gross x employee/employer rate, PLUS Additional Medicare surtax
                   above a filing-status-specific threshold (SINGLE/HOH/MFJ/MFS each differ)
3. FUTA (employer) = min(annual_gross, futa_wage_base) x rate, net of the standard federal credit
                   - ONLY when a real EmployerTaxProfile["SUI"] exists (never inferred)
4. SUI (employer)  = min(annual_gross, sui_profile.taxable_wage_base) x sui_profile.employer_rate_pct
                   - read from EmployerTaxProfile, NOT rate_map (statutory fact, not org policy)
5. State income tax = only when ctx.state_slabs resolved a real state-scoped bracket set
                   (California/New York); reciprocity (_resolve_us_reciprocity) can redirect
                   this to the RESIDENT state's own bracket set instead of the work state's
6. Local tax       = only when ctx.locality_rate resolved a real, manually-entered LocalityRate
                   (flat LST-style amount, or resident/nonresident rate_pct)
7. Federal income tax = _calculate_annual_tax_us - standard_deduction (filing-status-aware) then
                   progressive brackets, filing-status-filtered
   tds = federal_income_tax + state_income_tax + local_tax (broken out separately on the payslip,
   not one combined line mislabeled "Federal Withholding")
```

Every one of steps 3–6 explicitly resolves to zero rather than guessing when no real configuration exists — the standard's own "never infer an experience rate / never infer taxability" rule, applied consistently across FUTA credit, SUI, reciprocity, and locality.

Part 5 — Cross-Jurisdiction Comparison

CONCERN	INDIA	UK	USA
Frontend shape	1 page file + 5-file picker folder	1 page file + 1 shared modal	1 orchestrator + 3 component folders (17 files)
Regional/sub-jurisdiction concept	State-scoped Professional Tax (separate pack)	Scotland/Wales/NI — tax-code PREFIX picks region, never worksite	State income tax + locality (county/municipal) — worksite-based, with reciprocity override
Bracket mechanism	Cumulative marginal-rate sum (brackets touch at boundaries by design)	Same cumulative model; NI additionally supports banded-by-category rows	Same cumulative model; filing-status-tagable rows
Unique backend depth	Regime-dual rows (New/Old sharing one pack)	Tax-code interpretation (K-codes, flat-rate families)	5 extra tables (SourceArtifact/LocalityDataset/EmployerTaxProfile/ReciprocityRule/TaxabilityRule)
Known/flagged gaps	PF has no wage ceiling	No true cumulative (Week1/Month1) YTD basis yet	State brackets not yet filing-status-filtered (no data needs it today)

Generated from a live codebase read of ZoikoPayroll (zoiko-payroll-platform) — Super Admin Compliance + payroll engine, IN / UK / USA.